

Claude finished the response

You said: I have an exam question to answer here is the assignment file.

Final Exam: Required AI Prompt Instructions You may use AI during the final exam, but the transcript must show that you are doing the computational thinking. AI may help you implement, debug, and understand code, but it should not do the design thinking for you. For each exam problem, you must s

pasted

I have an exam question to answer here is the assignment file. Additionally I must utilize this framework to answer the question

Final Exam Question 2 — Plus/Minus Target Sum Search a state space by assigning + or - to each number to reach a target sum. Problem Given a list of numbers and a target sum, assign either + or - to each number so that the resulting expression equals the target. Each number must be used exactly once. If no assignment works, your program should report that no solution is possible. Example: Numbers: [1, 2, 3] Target: 4 One solution: $-1 + 2 + 3 = 4$ Input Format Your program should prompt the user as follows: Enter numbers on one line, separated by spaces Enter the target sum on a separate line Enter numbers: 1 2 3 Enter target: 4 State Space Pattern With n numbers, there are $2^{**} n$ possible sign assignments. 0 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 0 1 0 1 1 1 1 1 You may define: 0 $\rightarrow$ use - 1 $\rightarrow$ use + This shows the state space. A solution is one assignment that reaches the target. Your Task Read input from the user Generate sign combinations Check each combination Output a valid expression OR "No solution possible" Computational Thinking Expectations Identify inputs Define state Define transitions Explain invariants Explain how you explore the state space Allowed Python Concepts lists, loops, conditionals functions sum() Do NOT use recursion. Submission Requirements Item Description Program Solution code Test cases At least one success and one failure Comments Written by you AI Transcript Shows your thinking Using AI Upload this problem to AI Use the separate AI prompt template You must drive the design AI helps implement. You do the thinking.

Show more

8:55 AM

Claude responded: I'll help you work through this exam problem.

I'll help you work through this exam problem. Let me start by following the framework — I need to ask you about your design thinking before we write any code.

Let me set this up the way the exam requires. Here's how I'd suggest you approach this with your AI session, and I'll play the role of helping you think through it:

Before I help you implement anything, you need to explain your thinking on this problem. Let's go through the computational thinking expectations the exam lists:

1. Inputs — What is the program reading from the user, and in what form will you store it once read?

2. State — What changes as your program runs? For this problem, think about: what are you tracking as you try different sign combinations? What does a single "attempt" look like?

3. Transitions — How do you move from one state to the next? The hint about 2^{**n} combinations and the binary table (000, 100, 010, 110...) is a big clue. How does one row of that table become the next row?

4. Invariants — What must always remain true throughout your program? (Hint: think about the relationship between the number of signs and the count of numbers, or what each "0" or "1" must always represent.)

5. Data representation — How will you represent:

- the list of numbers?
- a single sign assignment (one row of that table)?
- the running sum you're checking against the target?
- the final expression string you output ($-1 + 2 + 3 = 4$)?

6. State space exploration — The problem says there are 2^{**n} combinations. How do you plan to generate all of them without using recursion? The binary counting pattern in the table is a hint — what does it look like if you count from 0 to $2^{**n} - 1$ in binary?

A few specific things to think about before you answer:

- Look at the table in the problem. The rightmost column changes slowest, the leftmost changes fastest. Does that match how you'd naturally count in binary, or is it reversed? Does it matter for correctness?
- When you find a valid combination, you need to output something like $-1 + 2 + 3 = 4$. The first number doesn't get a leading $+$ but gets a leading $-$ if negative. How will you build that string?
- What's your stopping condition? First solution found, or all solutions?

Take a shot at answering these in your own words — even rough answers are fine. Once you've laid out your design, I'll help you translate it into code that stays within the allowed concepts (lists, loops, conditionals, functions, `sum()`, no recursion).

You said: 1) the program is reading a list of #'s

1) the program is reading a list of #'s 2) as the program runs the order of values in order to hit the target change 3) there are only so many strings you can create given that you can only use each value once 4) you can only use each # once, the operations must sum to the target 5) as a string

8:58 AM

Claude responded: Good start — you're thinking in the right direction, but several of your answers are too vague or slightly off.

Good start — you're thinking in the right direction, but several of your answers are too vague or slightly off. Let me push back on each one so you can sharpen them. **This matters because the exam transcript needs to show *you* doing the design thinking.**

1. Inputs — "a list of numbers"

You're missing half of it. Re-read the input format. There are *two* separate inputs the user types. What are both of them? And when the user types `1 2 3`, what type is that initially, and what type do you need it to become?

2. State — "the order of values changes"

This isn't quite right. Read the problem again carefully — does the *order* of `[1, 2, 3]` ever change? Look at the example: `-1 + 2 + 3 = 4`. The numbers are still in their original order. What's actually changing from one attempt to the next?

Hint: look at the table in the problem statement. Each row is a *state*. What does each row represent?

3. Transitions — "there are only so many strings you can create"

That's a count, not a transition. A transition answers: *how do I get from one state to the next state?*

If your current state is 0 0 0 (meaning -, -, -), what's the next state? And the one after that? Look at the table — there's a pattern. Can you describe the rule that takes you from any row to the next row? (Hint: it's the same rule as counting up by 1 in binary.)

4. Invariants — "each # used once, operations must sum to target"

The "sum to target" part is **not** an invariant — that's the *goal*, only true for solutions. An invariant is something true on **every** attempt, even the failing ones.

"Each number used once" is closer. Try to list 2-3 things that are *always* true no matter which combination you're testing. For example: how many signs does every attempt have, compared to how many numbers? What are the only two values a sign can be?

5. Data representation — "as a string"

You only answered for one thing. I asked about four:

- The list of numbers → ?
- A single sign assignment (one row of the table) → ? (a string? a list? of what?)
- The running sum → ?
- The final output expression → a string, yes ✓

Be specific for each.

One more you didn't address: state space exploration.

The problem says there are $2^{**} n$ combinations and recursion is **not allowed**. So how are you going to generate all the rows of that table using only loops?

Think about this: if $n = 3$, there are 8 combinations. The numbers 0 through 7 in binary are 000, 001, 010, 011, 100, 101, 110, 111 — exactly the 8 rows you need. Does that suggest a strategy?

Take another pass at these. Your answers don't need to be polished — they need to be *specific*. Vague answers like "as a string" or "the order changes" won't show the examiner that you understood the structure of the problem.

You said: 1) the list and the target need to be inputted

1) the list and the target need to be inputted 2) the sign such as addition or subtraction changes 3) It's counting up by 1 4) positive and negative 5) it would be a list of numbers, yes it's got to be a single value, and the running sum must equal the target

9:00 AM

Claude responded: Better — you're getting clearer on signs and counting.

Better — you're getting clearer on signs and counting. But you're still skipping over things and being too brief. Let me push once more, and I'll be more specific about what's missing so you can lock it in.

1. Inputs ✓ (mostly)

Good — list and target. Now finish it: when the user types 1 2 3, Python receives "1 2 3" (a string). What two operations turn that into [1, 2, 3] (a list of integers)? And the target — what type does it need to be?

2. State — "the sign changes"

Yes! Now make it precise. The state at any moment is a **collection of signs**, one per number. So if numbers are [1, 2, 3] and the current state is [-, +, +], the expression being tested is -1 + 2 + 3.

Can you say it back in one sentence? *"The state is ___, one for each ___ in the input list."*

3. Transitions — "counting up by 1" ✓

Right idea. Now connect it to the problem: you're going to loop a counter `i` from `0` to `2**n - 1`. For each value of `i`, its **binary representation** gives you one row of the table — one sign assignment.

So the transition is: `i → i + 1`, which in binary flips bits in the standard counting pattern. This is your loop.

Question for you: how do you extract the individual bits of `i`? (There are a couple of ways — one uses `//` and `%` with 2, another uses Python's built-in `bin()`. Which feels more natural to you?)

4. Invariants — "positive and negative"

That's *one* invariant (each sign is one of two values). You need more. Here are three I'd expect on a strong exam answer — confirm each one is true:

- (a) The number of signs in any attempt always equals `len(numbers)`. (*True or false?*)
- (b) The original list of numbers never changes — only the signs do. (*True or false?*)
- (c) Each sign is either `+` or `-`, never anything else. (*True or false?*)

Just confirm these so I know you see them.

5. Data representation — getting closer

You said:

- List of numbers → list ✓
 - A single sign assignment → "a single value" ✗ — it can't be one value, it has to cover every number. If `numbers = [1, 2, 3]`, you need 3 signs. So a sign assignment is a **list** (or string) of length `n`. What goes inside it? The strings `+` and `-`? The integers `1` and `-1`? The bits `0` and `1`? Pick one and tell me why.
 - Running sum → you said "must equal target" — that's the *check*, not the *representation*. The running sum is just an integer. ✓ (implied)
 - Final output → a string like `"-1 + 2 + 3 = 4"` ✓
-

Skipped: state space exploration

You haven't answered this directly. Say it in one sentence: *"I will explore the state space by ____ from ____ to ____, and for each value I will ____."*

Pulling it together — answer these four to lock the design:

1. How do you convert "1 2 3" into [1, 2, 3]? (Two operations.)
2. For a sign assignment of length *n*, what will you store inside it — "+" / "-" strings, +1 / -1 integers, or 0 / 1 bits? Pick one.
3. Confirm invariants (a), (b), (c) above are all true.
4. Fill in: "I will loop a counter from ____ to ____, and for each value I will convert it to ____ to get the signs."

Once you answer these four, your design is complete and we can move to code.

You said: 1) the target needs to be a whole number

1) the target needs to be a whole number 2) the state is a collection of signs one for each # in the input list 3) // 4) confirmed

9:04 AM

Claude responded: Good — you've confirmed the core ideas.

Good — you've confirmed the core ideas. But you skipped two of my four lock-in questions, and I want to make sure your design is fully nailed down before we write any code. Let me consolidate.

Where you are:

✓ **Inputs:** list of numbers + target (a whole number / integer) ✓ **State:** a collection of signs, one for each number in the input list ✓ **Transitions:** count *i* from 0 upward; each *i* in binary gives one sign assignment; use // and % with 2 to extract bits ✓ **Invariants:** (a) number of signs = `len(numbers)`, (b) original list never changes, (c) each sign is + or - ✓ **Output:** string like "-1 + 2 + 3 = 4"

Two things still open — answer these and we move to code:

A) What do you store inside the sign assignment?

You have three reasonable choices:

- **Option 1: bits (0 and 1)** — store `[0, 1, 1]` meaning `-`, `+`, `+`. Easy to generate from binary, but you have to translate to `+1/-1` when summing and to `+/-` when printing.
- **Option 2: signed integers (+1 and -1)** — store `[-1, 1, 1]`. Easy to multiply with the numbers and sum directly. Still translate for printing.
- **Option 3: sign strings ("+" and "-")** — store `["-", "+", "+"]`. Easy to print, but you have to translate to numbers to sum.

Which one do you want to use? (There's no single "right" answer, but pick one and have a reason — that's the design decision.)

B) Convert "1 2 3" into `[1, 2, 3]` — what two operations?

Hint: one of them splits the string on spaces. The other converts each piece from a string to an integer. Name both.

Once you answer A and B, I'll give you the code, and then you'll explain it back to me and add your own comments.

You said: let's go with option 2

let's go with option 2 I'm not sure about converting the operations

9:05 AM

Claude responded: Good choice on Option 2 — signed integers +1 and -1 are the cleanest because you can directly multiply them with your numbers and use `sum()`.

Good choice on Option 2 — signed integers `+1` and `-1` are the cleanest because you can directly multiply them with your numbers and use `sum()`. That's a solid design decision.

On the conversion question — let me walk you through it since this is a Python mechanics question, not a design question.

When the user types `1 2 3` and you call `input()`, Python gives you the string `"1 2 3"`. You need `[1, 2, 3]` — a list of three integers.

Two operations:

1. `.split()` — a string method that breaks a string into a list of pieces, splitting on spaces by default.
 - `"1 2 3".split() → ["1", "2", "3"]`
 - Notice these are still *strings*, not integers yet.
2. `int()` — converts one string to one integer.
 - `int("1") → 1`
 - But `.split()` gave you a *list* of strings, so you need to apply `int()` to each one. You'll do this with a `for` loop, building up a new list.

So the flow is:

- `raw = input("Enter numbers: ") → "1 2 3"`
- `pieces = raw.split() → ["1", "2", "3"]`
- Loop through `pieces`, call `int()` on each, append to a new list → `[1, 2, 3]`

For the target, it's simpler — just one integer: `target = int(input("Enter target: "))`.

Your design is now complete. Let me restate it back so you can confirm:

Read a list of integers and a target integer. The state is a list of signs (+1/-1), one per number. Loop `i` from 0 to `2**n - 1`. For each `i`, extract `n` bits using `// 2` and `% 2`, translating bit 0 → -1 and bit 1 → +1. Build a sign list of length `n`, multiply elementwise with the numbers, and use `sum()` to check against the target. On the first match, build and print the output string. If the loop finishes with no match, print "No solution possible."

Confirm that matches your intent, and I'll write the code.

You said: let's do it

let's do it

9:06 AM

Claude responded: Here's the code.

Here's the code. Read it carefully — after you read it, I'm going to ask you to explain parts of it back to me, and you'll need to add your own comments.

python

```
def get_numbers():
    raw = input("Enter numbers: ")
    pieces = raw.split()
    numbers = []
    for piece in pieces:
        numbers.append(int(piece))
    return numbers

def get_target():
    return int(input("Enter target: "))

def build_signs(i, n):
    signs = []
    for position in range(n):
        bit = i % 2
        if bit == 0:
            signs.append(-1)
        else:
            signs.append(1)
        i = i // 2
    return signs

def compute_sum(numbers, signs):
    products = []
    for k in range(len(numbers)):
        products.append(numbers[k] * signs[k])
    return sum(products)

def build_expression(numbers, signs, target):
    result = ""
    for k in range(len(numbers)):
        if k == 0:
            if signs[k] == 1:
                result = result + str(numbers[k])
            else:
                result = result + "-" + str(numbers[k])
        else:
            if signs[k] == 1:
                result = result + "+" + str(numbers[k])
            else:
                result = result + "-" + str(numbers[k])
```

```

else:
    if signs[k] == 1:
        result = result + " + " + str(numbers[k])
    else:
        result = result + " - " + str(numbers[k])
result = result + " = " + str(target)
return result

def find_solution(numbers, target):
    n = len(numbers)
    total_combinations = 2 ** n
    for i in range(total_combinations):
        signs = build_signs(i, n)
        if compute_sum(numbers, signs) == target:
            return build_expression(numbers, signs, target)
    return "No solution possible"

def main():
    numbers = get_numbers()
    target = get_target()
    answer = find_solution(numbers, target)
    print(answer)

```

main()

Test it yourself with these cases:

- **Success:** numbers 1 2 3, target 4 → should give $-1 + 2 + 3 = 4$
 - **Success:** numbers 5 5 5, target 5 → should give something like $5 - 5 + 5 = 5$ (or $-5 + 5 + 5 = 5$, depending on which is found first)
 - **Failure:** numbers 1 2 3, target 100 → should give No solution possible
 - **Failure:** numbers 2 4 6, target 1 → should give No solution possible (all even numbers can't sum to an odd target)
-

Now your turn. Before you submit, you need to be able to explain these in your own words. Answer them:

1. **build_signs(i, n)** — explain how this function turns the integer **i** into a list of **+1**s and **-1**s. What do **i % 2** and **i // 2** do on each iteration?
2. **find_solution** — why does the loop go from **0** to **2 ** n**? What happens if a solution is found early vs. never found?
3. **build_expression** — why is the first number (**k == 0**) handled differently from the rest?
4. **Connecting to your design:**
 - o Where in the code is your **state** stored?
 - o Where do **transitions** happen (one state to the next)?
 - o Where are your **invariants** protected (sign list always length **n**, each sign always **+1** or **-1**)?

Answer these four, then I'll remind you that **you need to write your own comments on the code** — the exam explicitly says AI cannot comment it for you.

You said: 1) is this because python is 0 based so we must change the number?

1) is this because python is 0 based so we must change the number? 2) it doesn't make a difference 3) because it can't be 0?

9:09 AM

Claude responded: Your answers tell me you're not yet seeing what the code actually does.

Your answers tell me you're not yet seeing what the code actually does. That's okay — let me walk you through each one carefully, because **you cannot submit this exam until you can explain these in your own words**. The whole point of the framework is that the transcript shows your understanding.

1. **build_signs(i, n)** — what **i % 2** and **i // 2** actually do

Your guess about "0-based" isn't right. This has nothing to do with indexing — it has to do with **reading binary digits one at a time**.

Let's trace it concretely. Say **i = 5** and **n = 3**. In binary, 5 is **101**.

Iteration	<code>i</code> before	<code>i % 2</code> (bit)	sign appended	<code>i // 2</code> (<code>i</code> after)
1	5	1	+1	2
2	2	0	-1	1
3	1	1	+1	0

Result: `signs = [+1, -1, +1]`

What's happening:

- `i % 2` gives you the **last bit** of `i` in binary (the remainder when divided by 2 — either 0 or 1).
- `i // 2` **chops off that last bit** by integer-dividing by 2 (shifts everything one position to the right).

So the loop reads the binary digits of `i` one at a time, right-to-left, and translates each bit into a sign. Bit 1 → +1, bit 0 → -1.

This is the transition mechanism from your design. Each value of `i` from 0 to `2**n - 1` corresponds to one row of the sign table.

2. `find_solution` loop — "it doesn't make a difference" is wrong

It makes a huge difference. Let me re-ask in pieces:

(a) Why does the loop range from 0 to `2 ** n`?

Because the problem itself tells you there are exactly `2 ** n` possible sign assignments for `n` numbers. Each integer from 0 to `2**n - 1` represents one unique row of the sign table. If you stopped earlier, you'd miss combinations. If you went further, you'd repeat them.

For `n = 3`: $2^3 = 8$ combinations, and `range(8)` gives you 0, 1, 2, 3, 4, 5, 6, 7 — exactly the 8 rows.

(b) What happens if a solution is found early?

Look at this line: `return build_expression(numbers, signs, target)`. The word **return immediately exits the function**. So as soon as one valid combination is found, the loop stops and that expression is returned. We don't keep searching.

(c) What happens if no solution is ever found?

The `for` loop completes all 2^n iterations without ever hitting the `return` inside the `if`. Then execution falls through to the line *after* the loop: `return "No solution possible"`.

So the structure is: **return early on success, fall through on failure.**

3. `build_expression` — why is `k == 0` special?

Your guess "it can't be 0" isn't right either. The first number isn't special because of its value — it's special because of **how it's printed**.

Look at the expected output: `-1 + 2 + 3 = 4`

- The first number has **no leading space and no leading +**. If positive, it's just `1`. If negative, it's `-1`.
- Every number after the first has **+ or - in front of it** (with spaces).

If we treated `k == 0` the same as the others, we'd get `+ 1 + 2 + 3 = 4` — with a stray `+` at the start. That's ugly. So the first iteration is handled separately to avoid the leading operator.

4. Connecting to your design — I'll answer this one for you so you can see the mapping:

- **State** is stored in the `signs` list inside `build_signs`. Each call produces one state — one row of the table.
 - **Transitions** happen in `find_solution`'s `for i in range(total_combinations)` loop. Going from `i` to `i + 1` is the transition from one state to the next.
 - **Invariants are protected by the structure:**
 - `build_signs` loops exactly `n` times via `range(n)`, so the sign list is always length `n`. ✓
 - The `if bit == 0 / else` branch only ever appends `-1` or `+1`, nothing else. ✓
 - The `numbers` list is never modified — only read from. ✓
-

Now — go back and re-read the code with this explanation in hand. Trace `build_signs(5, 3)` on paper yourself. Run the program with `1 2 3` and target `4`.

Once you've done that, you need to:

1. **Add your own comments to the code.** The exam explicitly forbids AI-written comments. Write comments that explain *why* each part exists and how it relates to state, transitions, and invariants — not surface-level "adds 1 to total" comments.
2. **Be ready to explain in your own words** what `i % 2` and `i // 2` do, why the loop runs `2 ** n` times, and why the first number is printed differently.

Want to take another pass at explaining `build_signs` in your own words now that you've seen the trace? That's the part most likely to come up if anyone asks you about your code.

You said: can you give me suggestions for where my comments should go?

can you give me suggestions for where my comments should go?

9:11 AM

Claude responded: Yes — here's a guide to where comments belong and what each one should accomplish.

Yes — here's a guide to where comments belong and what each one should accomplish. I'm **giving you guidance on placement and purpose, not the comment text itself**. You write the actual words. That's what the exam requires.

Comment placement guide:

python

```
def get_numbers():
    # COMMENT 1: Explain what raw input looks like and why it needs converting.
    raw = input("Enter numbers: ")
    pieces = raw.split()
    # COMMENT 2: Explain why a loop is needed here (split gives strings, not ints).
    numbers = []
    for piece in pieces:
        numbers.append(int(piece))
    return numbers
```

```
def get_target():
```

```
# COMMENT 3: One short comment on why int() is needed around input().
return int(input("Enter target: "))
```

```
def build_signs(i, n):
    # COMMENT 4 (IMPORTANT): Explain what this function does at a high level —
    # turns an integer i into a sign assignment by reading its binary digits.
    # This is the heart of how you generate states.
    signs = []
    for position in range(n):
        # COMMENT 5: Explain what i % 2 extracts (the last binary digit / bit).
        bit = i % 2
        # COMMENT 6: Explain the bit-to-sign translation rule from your design
        # (0 means minus, 1 means plus — your Option 2 choice).
        if bit == 0:
            signs.append(-1)
        else:
            signs.append(1)
        # COMMENT 7: Explain what i // 2 does (chops off the bit you just read
        # so the next iteration reads the next bit).
        i = i // 2
    # COMMENT 8 (INVARIANT): Note that signs always has length n because the
    # loop runs exactly n times. This protects an invariant from your design.
    return signs
```

```
def compute_sum(numbers, signs):
    # COMMENT 9: Explain why you multiply numbers[k] * signs[k] —
    # this applies the sign to each number (your Option 2 choice pays off here).
    products = []
    for k in range(len(numbers)):
        products.append(numbers[k] * signs[k])
    # COMMENT 10: Note that sum() gives the value of the signed expression,
    # which is what you compare to the target.
    return sum(products)
```

```
def build_expression(numbers, signs, target):
    # COMMENT 11: Explain what this function produces (the human-readable
    # output string like "-1 + 2 + 3 = 4") and why it's separate from
    # compute_sum (one is for checking, one is for displaying).
    result = ""
    for k in range(len(numbers)):
        # COMMENT 12 (IMPORTANT): Explain WHY the first number is handled
```



```

# differently — no leading "+" or "-", but a leading "-" if negative.
if k == 0:
    if signs[k] == 1:
        result = result + str(numbers[k])
    else:
        result = result + "-" + str(numbers[k])
else:
    # COMMENT 13: Explain that every number after the first gets
    # "+" or "-" in front of it with spaces.
    if signs[k] == 1:
        result = result + " + " + str(numbers[k])
    else:
        result = result + " - " + str(numbers[k])
return result + " = " + str(target)

```

```

def find_solution(numbers, target):
    n = len(numbers)
    # COMMENT 14 (IMPORTANT): Explain why total_combinations is 2 ** n —
    # this is the size of the state space. Tie it back to the problem statement.
    total_combinations = 2 ** n
    # COMMENT 15 (TRANSITION): Explain that this loop is the state space
    # exploration. Each value of i is one state (one row of the sign table).
    # Going from i to i+1 is the transition.
    for i in range(total_combinations):
        signs = build_signs(i, n)
        # COMMENT 16: Explain the early-return — as soon as one valid sign
        # assignment is found, we stop searching and return the expression.
        if compute_sum(numbers, signs) == target:
            return build_expression(numbers, signs, target)
    # COMMENT 17: Explain that this line only runs if the loop completes
    # without finding any solution — i.e., we tried every state and none worked.
    return "No solution possible"

```

```

def main():
    # COMMENT 18: Brief comment explaining main() ties everything together —
    # input, search, output.
    numbers = get_numbers()
    target = get_target()
    answer = find_solution(numbers, target)
    print(answer)

```

main()

Priority comments — if you only nail a few, make sure these are strong:

The exam rubric rewards comments that show you understand **state, transitions, and invariants**. The most important comments are:

- **#4 and #5–7** — `build_signs` and the binary-reading mechanism. This is your *transition* logic.
- **#8** — the length-`n` invariant.
- **#14 and #15** — the `2 ** n` state space and the loop as exploration.
- **#12** — the first-number formatting rule (shows you understood the output structure).
- **#16 and #17** — early return on success vs. fall-through on failure.

If those seven are strong, the rest can be brief.

Style reminder from the exam doc:

Good: `# Update the running total after each item is processed.
total is part of the state because it changes during the
loop.` Weak: `# Add price to total`

So instead of `# multiply numbers and signs`, aim for something like `# Each product
applies one sign from the current state to its corresponding number;
this is how the sign assignment becomes a numeric value to compare to
the target.`

Write them in your own voice — short, plain, explaining *why* and *how it connects to the design*, not just *what*. Once you've drafted them, paste the commented code back to me and I'll check whether they show your understanding without rewriting them for you.

Claude is AI and can make mistakes. Please double-check responses.